RoSE-Opt: Robust and Efficient Analog Circuit Parameter Optimization With Knowledge-Infused Reinforcement Learning

Weidong Cao, *Member, IEEE*, Jian Gao, *Student Member, IEEE*, Tianrui Ma, *Student Member, IEEE*, Rui Ma, *Senior Member, IEEE*, Mouhacine Benosman, *Senior Member, IEEE*, and Xuan Zhang, *Senior Member, IEEE*

***Abstract*—Design automation of analog circuits has long been sought. However, achieving robust and efficient analog design automation remains challenging. This article proposes a learning framework, RoSE-Opt, to achieve robust and efficient analog circuit parameter optimization. RoSE-Opt has two important features. First, it incorporates key domain knowledge of analog circuit design, such as circuit topology, couplings between circuit specifications, and variations of process, supply voltage, and temperature, into the learning loop. This strategy facilitates the training of an artificial agent capable of achieving design goals by identifying device parameters that are optimal and robust. Second, it exploits a two-level optimization method, that is, integrating Bayesian optimization (BO) with reinforcement learning (RL) to improve sample efficiency. In particular, BO is used for a coarse yet quick search of an initial starting point for optimization. This sets a solid foundation to efficiently train the RL agent with fewer samples. Experimental evaluations on benchmarking circuits show promising sample efficiency, extraordinary figure-of-merit in terms of design efficiency and design success rate, and Pareto optimality in circuit performance of our framework, compared to previous methods. Furthermore, this work thoroughly studies the performance of different RL optimization algorithms, such as deep deterministic policy gradients (DDPGs) with an off-policy learning mechanism and proximal policy optimization (PPO) with an on-policy learning mechanism. This investigation provides users with guidance on choosing the appropriate RL algorithms to optimize the device parameters of analog circuits. Finally, our study also demonstrates RoSE-Opt's promise in parasitic-aware device optimization for analog circuits. In summary, our work reports a knowledge-infused BO-RL design automation framework for reliable and efficient optimization of analog circuits' device parameters. Code implementation of our method can be found at https://github.com/xz-group/RoSE.**

***Index Terms*—Analog integrated circuits, design automation, Bayesian optimization, deep reinforcement learning, graph neural networks.**

Manuscript received 27 January 2024; revised 7 May 2024 and 10 July 2024; accepted 22 July 2024. Date of publication 29 July 2024; date of current version 22 January 2025. This work was supported in part by the National Science Foundation under Grant CCF-1942900. The work of Weidong Cao was supported in part by Internship at Mitsubishi Electric Research Laboratories. This article was recommended by Associate Editor H. E. Graeb. *(Weidong Cao and Jian Gao contributed equally to this work.) (Corresponding author: Weidong Cao.)*

Weidong Cao is with the Department of Electrical and Computer Engineering, The George Washington University, Washington, DC 20052 USA (e-mail: weidong.cao@gwu.edu).

Jian Gao and Xuan Zhang are with the College of Engineering, Northeastern University, Boston, MA 02115 USA.

Tianrui Ma is with the Department of Electrical and Systems Engineering, Washington University in St. Louis, St. Louis, MO 63130 USA.

Rui Ma is with the Department of mmWave PA Systems, pSemi a Murata Company, San Diego, CA 92121 USA.

Mouhacine Benosman is with the Department of Data Analytics, Mitsubishi Electric Research Laboratories, Cambridge, MA 02139 USA.

Digital Object Identifier 10.1109/TCAD.2024.3435692

I. Introduction

INTEGRATED circuit (IC) technology advances human society by powering numerous applications and infrastructures with microelectronic chips of a small footprint. Recent advances in deep learning have shown great promise in transforming modern IC design workflows [1], [2], [3]. By formulating each design stage as a learning problem, machine learning techniques can significantly shorten IC development cycles compared to conventional electronic design automation (EDA) tools. For example, Google [2] and Nvidia [3] have shown that deep learning methods can improve design efficiency by an order of 100× at certain stages of the digital IC design flow, such as floor planning and power estimation. Analog circuit is an essential type of circuit that bridges our physical world with the digital information realm [4], [5], [6], [7], [8]. Yet, unlike digital ICs that benefit from well-established conventional EDA tools or emerging efficient learning-based design automation methods, analog circuits continue to rely on onerous human efforts and lack effective EDA techniques at all stages [1], [9].

The prelayout design of analog circuits can be represented as a parameter-to-specification (P2S) optimization problem. Given the topology of a circuit, the goal is to find optimal device parameters (e.g., width and finger number of transistors) to meet the desired specifications [e.g., power and bandwidth (BW)] of the circuit. This problem is challenging due to several factors. First, it involves searching for parameters of diverse devices in a large design space. The complexity grows exponentially with an increase in both design parameters and circuit specifications [4], [5]. Second, the actual interactions between the device parameters and the circuit specifications are complicated [1], [9], depending on multiple variables, such as circuit topology, variations in process, voltage, and temperature (PVT) and post-layout parasitic effects. There are no exact analytical rules to follow,

1937-4151 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

which worsens the search process. Conventionally, human designers use critical domain knowledge, such as circuit topologies and couplings between circuit specifications, to manually derive the device parameters. In particular, a human designer exerts an intense effort to obtain empirical equations between the device parameters and the circuit specifications based on a simplified circuit topology. However, despite the simplification, tens and even hundreds of iterative fine-tunings are still required to ensure the accuracy and reliability of the design.

During the past several decades, there have been enormous explorations on automating the design of analog circuit device parameters. These methods generally fall into two categories, knowledge-based techniques and optimization-based techniques. Knowledge-based techniques are designer-centric [10], [11], [12]. They customize the design steps for specific circuits based on domain knowledge and embed them into procedural scripts that mimic the actions of designers. These scripts allow designers to have full control over the modification and debugging of circuits to guarantee design reliability. However, design efficiency is significantly thwarted, because designers, acting as optimization agents, are required to frequently interact with procedural scripts. In contrast, optimization-based techniques are algorithm-centric. They consider each step of analog circuit design as a black-box optimization problem and apply optimization methods, such as Bayesian optimization (BO) [1], genetic algorithms [13], and emerging machine learning algorithms [9], [14], [15], [16], [17], [18]. These algorithms can be run quickly to complete the design of an analog circuit with high efficiency. Unfortunately, due to the absence of knowledge from experienced designers, the reliability of the design is not guaranteed, e.g., device parameters are not robust to various nonidealities. These defects limit the efficiency and reliability of state-of-the-art analog design automation techniques.

To bridge this gap, we propose a learning framework, RoSE-Opt, to achieve robust and efficient analog circuit parameter optimization by synergizing domain knowledge of analog circuits and learning algorithms. Analog circuit design strongly relies on domain knowledge, such as circuit topology, couplings between circuit specifications, and PVT variations; thus, without adequately considering these key domain knowledge in building learning-based design automation frameworks, the device parameters discovered by the algorithm are prone to suffer from inferior reliability issues due to various nonidealities. Our previous work [19] follows this principle and has explored the integration of this key domain knowledge into the design framework. It also exploits a two-level optimization method by integrating BO and reinforcement learning (RL) to improve sample efficiency.

In this article, we propose the RoSE-Opt framework which advances the state-of-the-art method [19]. In particular: ❶ We study its ability to consider device parasitic in parameter optimization. A direct mapping of an analog circuit schematic with correctly sized devices into a physical layout can lead to performance degradation, mainly due to parasitics from metal wires and electromagnetic effects, and needs tens of iterations between the two for fine-tuning. This extended work demonstrates the promise of RoSE-Opt in addressing this problem. ❷ At the algorithm level, we thoroughly study the performance of different RL optimization algorithms, such as deep deterministic policy gradients (DDPGs) with an off-policy learning mechanism and proximal policy optimization (PPO) with an on-policy learning mechanism, to provide users with useful guidance in choosing appropriate RL algorithms for device sizing. ❸ Finally, we showcase a Pareto optimization example, i.e., optimizing the figure-of-merit (FoM) of a circuit. Our framework can facilitate the identification of the optimal Pareto frontier. In summary, we present a holistic BO-RL-based design automation framework to perform the P2S optimization tasks of analog circuit design with high robustness and efficiency. We make following key contributions.

1) This article proposes a comprehensive BO-RL-based design automation framework, RoSE-Opt. Our learning framework explores and exploits both the domain knowledge of analog circuit design and the strong optimization ability of design automation algorithms.
2) In addition, we study its effectiveness in the scenarios of parasitic-aware device parameter optimization.
3) We thoroughly characterize the performance of different RL optimization algorithms (i.e., DDPG versus PPO) to provide users with useful insights in choosing appropriate RL algorithms for device parameter optimization.
4) Experimental evaluations on benchmarking circuits show that our framework achieves 7.9×∼12× improvement in training sample efficiency and a significant improvement in the design success rate and efficiency compared to the state-of-the-art methods for the same problem. In Pareto optimization, our framework identifies the optimal Pareto frontier by minimizing power consumption and maximizing the gain BW product (GBW), all within a minimal number of simulations.

II. Background and Related Work

In this section, we first review the basics of BO and RL. We then introduce the key domain knowledge that human experts commonly consider when addressing the P2S problem. Finally, we discuss existing design automation methodologies for analog circuits.

A. Bayesian Optimization

BO proves to be a valuable framework to address challenging black-box optimization problems that involve costly function evaluations. BO's fundamental concept is to construct an inexpensive surrogate model, such as a Gaussian Process (GP), by leveraging actual experimental data. This surrogate model incorporates prior knowledge or beliefs about the objective function, which is then used to make informed decisions in the process of selecting a sequence of function evaluations through the use of an acquisition function, such as expected improvement (EI). It also balances exploration and exploitation. Exploration allows for a broader exploration of the search space, potentially discovering better solutions, while exploitation focuses on exploiting the known promising areas to optimize the current best solution. Balancing these

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

two aspects is crucial to finding better solutions and refining the best solution.

Given an arbitrary function $f(\vec{x})$ for maximization, there are several steps to follow for BO. *Step 1—Initial Sampling*: Here, a limited set of sample points is randomly selected. *Step 2—Initializing the Model*: These points in Step 1 are used to calculate a surrogate function. *Step 3—Iterating*: In particular, the acquisition function is first used to get the next point; then, the surrogate function is re-evaluated; third, the surrogate function is verified to see if it remains stable or if the variance falls below a predetermined threshold, or if $f(\cdot)$ is exhausted, depending on the specific design objective.

BO is well suited to optimizing hyperparameters of many classification and regression models. It is also used for circuit design automation [1], [20], [21], [22] (e.g., the P2S optimization tasks for analog circuit design [1]).

B. Reinforcement Learning

RL is a machine learning method related to how intelligent agents take actions in an environment to maximize cumulative returns based on states. There are five essential elements in an RL problem: 1) agent; 2) action; 3) state; 4) reward; and 5) environment. The "Agent" is the learner and the decision maker who learns experiences from a training process and makes decisions based on observations (states) from the environment. The "Action" is a set of operations that the agent can perform. The "State" is a representation of the current environment (i.e., observations) in which the agent is staying. It can be observed by the agent and contains all relevant information about the environment that the agent needs to know to make a decision. The "Reward" is a scalar value returned by the environment after the agent takes an action in a state. It is used to evaluate and guide the actual learning behavior of the agent. The "Environment" is the physical world in which the agent operates.

In each episode, an agent starts from an initial state, then observes the state o_k and takes an action a_k based on a policy. Meanwhile, the environment updates a reward r_{k+1} for that particular action and enters a new state o_{k+1}. The agent iterates through the episode in multiple steps, accumulating the reward at each step to obtain the final return. With multiple episodes, the RL agent improves its decision quality and finds the best policy to maximize the return. Such a policy would be deployed for practical tasks, i.e., the agent follows the trained policy to finish a given task.

RL algorithms have been extensively applied to many problems, such as game playing [23], robotics [24], computer vision [25], and natural language processing [26]. RL has also been used to automate the design of ICs, such as the placement of the digital IC chip [27] and the P2S optimization of analog circuits [17], [18], [28].

C. Key Domain Knowledge of Analog Circuit Design

At the prelayout stage, there are many considerations to be taken by human experts to select reliable device parameters and meet the design goals. These considerations are the domain knowledge, and we introduce the major ones that are commonly used by human experts when they tackle the P2S optimization tasks, as shown in Fig. 1.

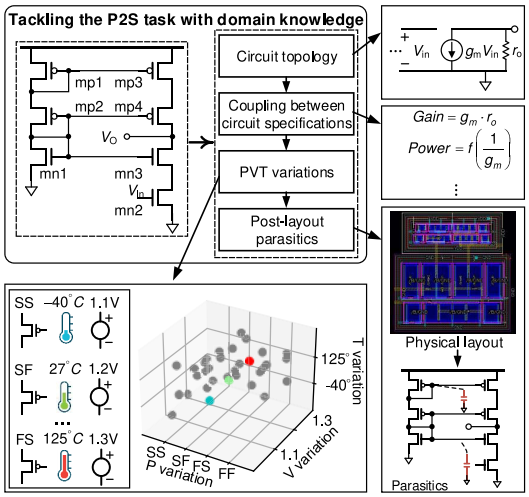


Fig. 1. Illustration of a manual design flow to tackle the P2S optimization tasks with human domain knowledge.

1) Circuit Topology: When human experts manually find the optimal device parameters, they first construct the circuit small-signal model from the circuit topology, based on which they obtain empirical equations that connect the device parameters to the circuit specifications. With these equations, device parameters can be derived by hand.

2) Couplings Between Circuit Specifications: Due to design tradeoffs, circuit specifications often depend on each other. For example, in the design of operational amplifiers (Op-Amps), energy efficiency often trades off with gain; that is, a higher-amplification gain requires a larger transconductance, which, however, demands more power consumption and results in lower-energy efficiency. Therefore, in a conventional manual design process, human experts use tens and even hundreds of iterative fine-tunings to find a group of proper device parameters to satisfy all circuit specifications.

3) PVT Variations: To ensure the robustness of analog circuits in different harsh environments, a key design consideration is to minimize the influence of variations in process (P), voltage (V), and temperature (T). Process variation represents the deviation of the manufactured devices from their ideal specification due to manufacturing errors. It includes typical N-type transistor/typical P-type transistor (TT), fast N-type transistor/fast P-type transistor (FF), slow N-type transistor/slow P-type transistor (SS), slow N-type transistor/fast P-type transistor (SF), and fast N-type transistor/slow P-type transistor (FS). Voltage and temperature variations are due to uncertain ambient changes. Typical deviation of the supply voltage is $\pm 10\%$ from its nominal value V_{DD}; and typical range of the environmental temperature for circuits is $[-40, 125]^{\circ}$C. A single PVT corner is a combination of P, V, and T from their varying ranges. All these variations are unavoidable and can cause the circuit performance degeneration compared to its nominal case, i.e., $\{\mathrm{TT}, V_{\mathrm{DD}}, 25\ ^{\circ}\mathrm{C}\}$. Manual experts have to look for robust device parameters to achieve the design goal in all PVT corners.

4) Parasitic Effects of Physical Layouts: A complete flow of analog circuit design includes the schematic design and

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

the physical layout design. The conversion of an analog circuit schematic, with the correctly sized components, into a physical layout can cause performance degradation due to the parasitic effects of metal wires and electromagnetic couplings. Experienced human designers often make efforts to adjust the device parameters to ensure that the post-layout simulation meets the desired objectives.

D. Existing Design Automation Methodologies

Various design automation techniques have been proposed for the P2S tasks of analog circuits in recent years. They generally fall into two categories: 1) knowledge-based techniques and 2) optimization-based techniques. Knowledge-based techniques, such as BAG [10], are designer-centric. They tailor the design steps for specific circuits with domain knowledge and embed these steps into the procedural scripts that mimic designer actions. These scripts provide designers with complete control over circuit modifications and debugging to ensure design reliability. Yet, these approaches notably affect design efficiency, as they demand frequent interactions between designers and procedural scripts, with designers playing the role of optimization agents. On the contrary, optimization-based methods, such as BO [1], geometric programming [29], genetic algorithms [13], and modern machine learning approaches [9], [14], [15], [16], [17], [18], [28], are centered on algorithms. They treat each step in a circuit design as a black-box optimization problem and can swiftly perform optimization procedures to complete a circuit's design with high efficiency. Unfortunately, the lack of knowledge from seasoned designers means that design reliability, such as the robustness of device parameters to nonideal conditions, is not assured. These limitations significantly impact the widespread applications of state-of-the-art analog design automation techniques, as they are unable to achieve both high-design efficiency and reliability. Thus, essential to advance analog design automation is to adequately incorporate design knowledge into optimization algorithms to ensure design reliability while maintaining high-optimization efficiency.

Learning-based optimization methods have recently emerged. They show higher-design efficiency and scalability in handling the P2S task compared to classical optimization algorithms, such as BO [1], geometric programming [29], and genetic algorithms [13], that are limited to ad-hoc tasks and tackling simple circuits. As an example, supervised learning methods [9], [14], [15], [16] have been used to learn the complicated relations between device parameters and circuit specifications. Once trained, they adopt one-step inference to predict optimal device parameters for given design goals. Nonetheless, these supervised learning methods cannot guarantee a high-design success rate and suffer from weak generalization abilities [9], [14], [15], [16] due to their inherent approximation errors. On the other hand, RL methods [17], [18], [28] learn an optimal policy from the state space of circuit specifications to the action space of device parameters, which solves a quasi-dynamic programming problem. They often use multiple sequential decision steps to find the optimal device parameters rather than just using one-step prediction, thus achieving a higher-design success rate and better generalization abilities than supervised learning methods [9], [14], [15], [16]. However, none of them has taken into account sufficient domain knowledge of analog circuit design in the optimization loop, leading to low-design reliability.

In this work, we propose a learning-based framework, RoSE-Opt, to achieve efficient and reliable parameter optimization of analog circuit devices by harnessing the synergy between the knowledge of human designers and RL algorithms (elaborated in Section III). In particular, we leverage the rapid convergence of BO to identify an optimized starting point, significantly improving the sampling efficiency of the primary RL agent during its learning phase.

III. RoSE-Opt Framework

In this section, we introduce the proposed RoSE-Opt framework that automates P2S tasks. We start with the formulation of the problem. Then, an overview of the RoSE-Opt framework is presented, followed by an elaboration of the BO vanguard. Finally, we introduce five essential parts of the RL backbone and show how key domain knowledge is incorporated into the framework.

A. Problem Formulation

We target the P2S problem with a given circuit under stringent PVT variations and parasitic effects of physical layout, formulated as

$$\begin{aligned} &\min_{s} \ f(s,g), \\ &\text{s.t.} \ \ s = F(x), \ \ \text{where} s \in \mathbb{R}^{i\times j}, \ \ x \in S_P, \ \ g \in S_G. \end{aligned} \tag{1}$$

Here, the function $f(s, g)$ represents the difference between the circuit specifications s and the design goal g. For example, for Op-Amps, there are four main circuit specifications, i.e., gain (G), power consumption (P), phase margin (PM), and BW. $F(\cdot)$ is a circuit simulator environment to obtain circuit specifications s based on a set of device parameters x, e.g., width and finger numbers of transistors. s is essentially a matrix where i represents the specification type (e.g., gain) and j represents a PVT corner. Therefore, $s \in \mathbb{R}^{4\times 16}$, assuming 16 PVT corners for the design of Op-Amps. The set of device parameters x is restricted by the design space S_P. The design goal g is restricted to a reasonable sampling space S_G that the circuit can achieve. Our objective is to minimize $f(s, g)$ by efficiently looking for a group of optimal device parameters so that the circuit specifications can meet an arbitrarily given group of design goals under all PVT variations. Considerations of parasitic effects are discussed in Section V-B as it is considered during the deployment stage rather than the training stage.

B. Framework Overview

We explore the synergy of BO and RL to achieve robust and sampling-efficient device parameter optimization. Fig. 2 shows the overview of the proposed RoSE-Opt framework, which contains two parts: 1) a BO Vanguard and 2) an RL Backbone. BO is a well-known optimization algorithm that often achieves the fastest convergence [1] to an optimum (or suboptimum) for a given design goal, compared to other

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

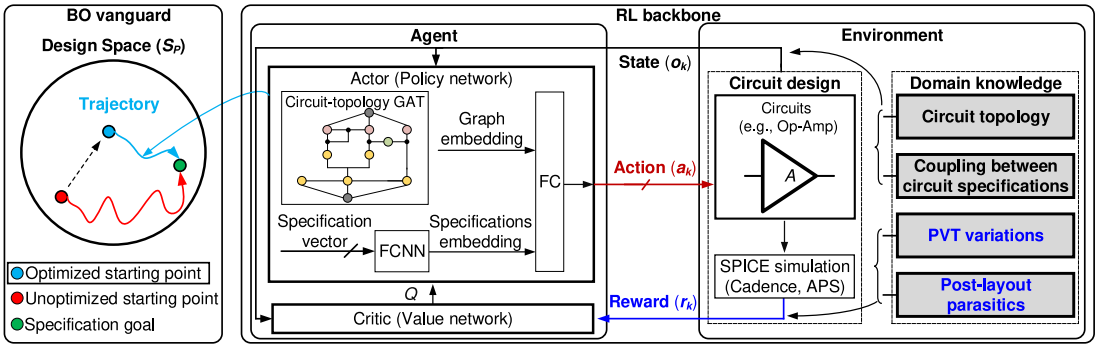


Fig. 2. Overview of our RoSE-Opt framework for automated design of analog circuits by complementing BO vanguard and RL backbone. At its core, the framework leverages BO's rapid convergence to identify an optimized starting point for RL, significantly enhancing its sample efficiency throughout the learning or optimization process. This strategy combines the efficient exploration capabilities of BO with the robust optimization power of RL to ensure both design robustness and efficiency. The RL backbone is based on an actor-critic method. The environment consists of a netlist of an analog circuit with a given topology, a circuit simulator, and a data processor. At each time step k, the agent automatically produces an action a_k to update device parameters with its policy network according to the state o_k and then receives the reward r_k from the environment. Our customized policy network is composed of a circuit topology-based GNN (i.e., GAT) and an FCNN.

optimization techniques [13], [30]. However, it needs to be restarted from scratch if the given design goal is changed and is often limited to tackling simple circuits with fewer dimensions.

In contrast, well-trained RL agents can reach general design goals without retraining based on a deployment trajectory from a starting point. Unfortunately, for robust analog circuit design, which is a more complex problem, RL methods demand more data points from time-consuming circuit-level simulations (i.e., PVT simulations) to sufficiently explore design space, leading to a low-sampling efficiency toward the convergence. With this key insight in mind, we propose to leverage BO as a vanguard to first search coarsely for a suboptimal starting point (i.e., initial device parameters) for our RL agent. On this basis, the RL agent can be trained to find optimal solutions with much fewer interactions with time-consuming circuit-level simulations, improving the sampling efficiency. As conceptually shown in the left subset of Fig. 2 (i.e., BO vanguard), an optimized starting point can help the RL agent reach design goals with a shorter trajectory compared to a randomly selected one. Hence, it can guide the RL agent to converge faster with fewer training data.

The RL backbone has five essential components similar to typical RL methods (refer to Section II-B): 1) reward; 2) action space; 3) state space; 4) environment; and 5) agent. To train an excellent RL agent for a given task, there are several critical factors to pay attention to. The first is to develop a comprehensive environment that could expose environmental information about the task to the RL agent as much as possible. The second is to capture sufficient exposed observations (states) relevant to the task from the environment into the learning loop. The third is to design a proper reward function that is closely related to the optimization goal and stimulates the learning of the RL agent. Finally, the agent (i.e., policy network) itself should be an expressive and powerful model that can distill the underlying domain knowledge and generalize it to unseen design goals. With these key factors in mind, we briefly introduce here: 1) How to develop the majority of these five components below and 2) How to distribute the domain knowledge of analog circuit design presented in Section II-C to the reward function and the customized policy network. More details will be discussed in Section III-D.

Comprehensive Environment: First of all, we develop a thorough circuit design environment for the P2S task, which includes the full circuit netlist of the given task and commercial simulation/verification tools (e.g., Cadence Spectre) for simulating circuit specifications (under PVT variations) and extracting post-layout parasitics.

Sufficient Observations: Second, the circuit design is a dynamic process that needs sufficient observation from its simulation environment. In particular, for a robust analog design, full PVT corner simulations are essential to ensure training stability. Thus, our RL's state consists of dynamic intermediate circuit specifications from all PVT corners and the corresponding circuit topology/device parameters.

Custom Reward Function: Third, PVT variations and parasitic of post-layouts affect the circuit specifications, which are directly related to the optimization goal. We thus use a custom reward function to take into account PVT variations (refer to (3) for details). By infusing key domain knowledge (e.g., PVT variations) into the RoSE-Opt framework through the reward function, an excellent RL agent can be trained and make good decisions to search for reliable device parameters that meet the design goals.

Customized Expressive Policy Network Model: Lastly, we customize a policy model architecture to enhance its expressiveness by taking in sufficient observations. In particular, we tailor a novel circuit topology-based GNN and an FCNN to incorporate into the learning loop of an RL agent. This policy network can effectively capture the essential physical features (e.g., device parameters and interactions) embedded in a circuit graph with the GNN and extract the couplings (i.e., design tradeoffs) between circuit specifications with the FCNN, which better models the relations between the circuit parameters and the design targets.

During the training of an RL agent, in each episode, the agent starts from an initial state o_0 with a group of initial device parameters optimized by the BO vanguard and a group of randomly sampled desired specifications g from sampling space S_G. The end of an episode occurs when the design goals are achieved or a predefined maximum step T is reached. At

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

each time step k, the agent begins using a neural network to observe a state o_k and take discrete action a_k based on the probability distribution of the output of the neural network. The agent then arrives in a new state o_{k+1} and receives a reward r_k from the environment. The discrete action a_k can update simultaneously all the device parameters for the given circuit. The agent iterates through the episode with multiple steps and accumulates the reward at each step until the end of the episode. In the next episode, the agent randomly samples another design goal g from the sampling space S_G and resets the parameter back to the starting point o_0. Then repeat the same process again. Once the policy network is well trained, we can save the weight of the neural network for deployment. During the deployment, since the weight has already been trained, the agent uses only the actor to take actions based on the state it observed. The purpose of the deployment part is to show the generalization capability of our trained policy network to different specifications without retraining like BO. Thus, we are interested to see how many specifications the decision policy can reach within the predefined maximum step T and what is the average deployment length for each run.

A key point is that BO is only required once in our framework if the sampling space S_G of the design goals and the design space S_P of each circuit device are defined. The RL agent then uses the same optimized starting point o_0 during all training episodes and the deployment stage. Note that in the context of robust device sizing, the designs of both the BO vanguard and the RL backbone are nontrivial and are elaborated in the following.

C. BO Vanguard

We rely on BO to find an optimized initial search point for our RL agent to improve its sampling efficiency during training. However, a crucial initial question is how to define such an optimized starting point. This starting point should not only speed up the design for a specific set of goals but should also help, in general, to efficiently design any arbitrary group of design goals from the entire sampling space S_P.

To solve the problem, we think that from this starting point, the RL agent should generally take the least deployment steps to achieve a general design. Thus, we let the device parameters found by BO that achieve as closely as possible the arithmetic mean of the maximum/minimum of each design goal in the entire sampling space S_G, i.e., $(PM_{\max} + PM_{\min})/2$, $(G_{\max} + G_{\min})/2$, $(BW_{\max} + BW_{\min})/2$, and $(P_{\max} + P_{\min})/2$, be the starting point of our RL agent. Here, taking a two-stage Op-Amp as an example, G, B, PM, P are the circuit specifications, i.e., gain (G), BW (B), PM, and power consumption (P). Another important question is how to balance the simulation budget between BO and RL. Our strategy is to monitor the improvement of the reward function every 10 iterations and stop BO optimization if the improvement falls below a specified threshold (i.e., 0.002). Users can also adjust the maximum number of simulations assigned to BO, typically set to 50, to ensure efficient resource management. BO is stopped when either is reached. Both hyperparameter values are set based on our empirical study.

We use a typical set-up of BO to search for the optimized starting point, which includes two essential parts: 1) the surrogate model and 2) the acquisition function [1]. The whole optimization depends on how accurately the surrogate model estimates the black-box function. In particular, we adopt the widely used GP model as our surrogate model to predict the underlying black function with uncertainty. We use a Monte-Carlo-based EI acquisition function to balance exploration and exploitation during the optimization by offering the next sampling point as below

$$\begin{aligned} \mathrm{EI}(X) &\approx \frac{1}{Z}\sum_{i=1}^{Z} \max_{j=1,\ldots,n} \left\{\max\left(\xi_{ij} - f(s,g)_{\text{best}}, 0\right)\right\} \\ \xi_i &\sim \mathbb{P}(f(X)|\mathcal{D}). \end{aligned} \tag{2}$$

Here, the expectation, $\mathrm{EI}(X)$, is computed by approximating the integrals over the posterior distribution over Z points using Monte-Carlo sampling. $\mathbb{P}(f(X)|\mathcal{D})$ is the posterior distribution of our function $f(s, g)$ at X where $X = (x_1, \ldots, x_n)$ from the sampling in our design space S_P. $\mathcal{D}$ is our data set. The parameter ξ determines the amount of exploration during optimization.

D. RL Backbone

The five key components of the RL backbone are detailed below.

1) Variation-Aware Reward Function: We connect the objective in (1) to our reward function so that our RL agent can be directly optimized considering PVT variations. Particularly, the reward r_k at each time step k is designed by taking PVT variations into consideration, i.e.,

$$\begin{aligned} r_k &= \mathrm{Mean}\left(\sum_{j=0}^{j=M-1} r_j\right);\ \text{if } \exists j \in [0,\ M-1],\ r_j < 0; \\ \text{or } r_k &= R,\ \text{if } \forall j \in [0,\ M-1],\ r_j = 0. \end{aligned} \tag{3}$$

Here, $r_j = \sum_{i=0}^{N-1} w_i \times \min\{(s_i^j - g_i)/(s_i^j + g_i), 0\}$ is the subreward of the jth corner, calculated based on a weighted sum of the normalized difference between the ith intermediate circuit specification of the jth corner s_i^j and the ith design goal g_i. All types of circuit specifications are equally important, i.e., $w_i = 1$. M represents the number of PVT corners and N indicates the number of circuit specifications. In order not to over-optimize the specification, we set the upper bound of r_j to be 0. Only when the circuit specifications in all PVT corners meet the design goal, a large stimulated reward of $R = 10$ is given to encourage the agent for the successful design; otherwise, the reward in each time step is the average of subrewards of all PVT corners. Finally, the accumulative reward for a training episode is $R_{s,g} = \sum_{k=1}^{T} r_k$, where T is a predefined maximum step for an episode. Intermediate circuit specifications matrix s is obtained from our high-fidelity simulation environment $F(\cdot)$ based on the updated device parameters x at each time step. Therefore, our reward is a direct measurement from the circuit simulator, which can help train a high-quality RL policy network.

2) Fine-Grained Action: Inspired by human designers who rely on multiple fine-grained tuning steps to find optimal

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

device parameters, we use a discrete action space to tune device parameters. For each tunable parameter x of a device (e.g., the width and finger number of transistors, the capacitance of capacitors), there are three possible actions: 1) increasing $(x + \triangle x)$; 2) keeping $(x + 0)$; or 3) decreasing $(x - \triangle x)$ the parameter, where "$\triangle x$" is the smallest unit used to update the parameter within its bound $x \in [x_{\min}, x_{\max}]$. With the total parameters of M devices, the output of the policy network is a matrix of probability distribution $M \times 3$ in any state where each row corresponds to a parameter. The action is taken based on the probability distribution.

3) Circuit Physics-Related State: RL belongs to representation learning. Capturing adequate state information from the environment is key to training an excellent RL agent. We leverage the domain knowledge of analog circuits, i.e., intermediate circuit specifications and circuit topology, as our state, which covers the most essential observations from a circuit design environment. In particular, we take care of intermediate circuit specifications in all PVT corners, in contrast to the previous work [31] which only considers partial PVT corners. We create a state vector to represent the intermediate circuit specifications. To better use the observations of the circuit itself, we use a graph $G(V, E)$ to model the circuit according to its topology, where each node in the set V is a device and the connections between the devices constitute the edge set E.

Fig. 2 shows the mapping between the circuit topology and a graph taking a two-stage Op-Amp as an example. For a circuit with n nodes, the state of the ith node is defined as its node feature $(t, \vec{p})$, where t is the binary representation of the type of node and $\vec{p}$ is the parameter vector of the node. Note that the parameters of the circuit device reflect the physical information of the circuit. For transistors, the parameters are the width (x_W) and the number of fingers (x_F). For capacitors, resistors, or inductors, the parameters are scalar values (e.g., capacitance, resistance, or inductance) of each device. For example, for a circuit with five different types of devices, the state of a N type transistor can be expressed as $[0, 0, 1, x_W, x_F]$.

4) SPICE Simulation Environment: In our work, a high-fidelity circuit design environment with PVT variations and post-layout parasitics is used. It consists of the netlist of a given analog circuit, a commercial circuit simulator, e.g., cadence spectre for CMOS analog circuits or keysight advanced design system (ADS) for RF power circuits, and a data processing module (DPM). As shown in Fig. 2, the simulator obtains intermediate circuit specifications at each time step. The DPM then deals with the simulated results to return a reward to the agent using (3). Meanwhile, it also updates the device parameters to rewrite the circuit netlist based on the actions of the agent (i.e., policy network).

Previous methods [31] assume that the circuit simulation time scales linearly with the number of simulations, i.e., the simulation time for 16 PVT corners is 16× of the one with a single PVT corner. We use the Cadence Spectre accelerated parallel simulator (APS) to accelerate our simulation. At each time step, we obtain circuit specifications for all PVT corners. Compared to a single PVT corner time, the batch simulation manner for 16 PVT corners only brings 0.17× time overhead as compared to a single PVT simulation. In other words, our circuit environment can achieve a sampling efficiency of at most 14× when collecting data points during training compared to previous RL methods. With this co-design loop, we are able to simultaneously achieve both high-sample efficiency and robust design by taking advantage of BO, RL, and the simulation environment.

5) Circuit-Aware Policy Network: We adopt an Actor-Critic method [32] to design our agent. To capture sufficient observations from the environment to the learning loop, we tailor a novel multimodal policy network for the Actor by integrating several unique features, as shown in Fig. 2.

1) We customize a new network architecture consisting of two different networks, a GNN based on the circuit topology and an FCNN. It is termed a GNN-FC-based policy network. Specifically, the GNN is used to distill the underlying physics (e.g., device types, parameters, and interactions) of a circuit graph into a low-dimensional vector embedding. The FCNN utilizes the design objectives and intermediate circuit specifications across all PVT corners as input to reveal their interconnected relationships, i.e., design tradeoffs. The graph embedding and the FCNN embedding are then concatenated for further processing by the final FC layers to update the actions. The value network (Critic) has the same architecture as the policy network except for the last layer. It is used to evaluate the quality of the actor's decision by giving an estimate of the expected reward, Q, for the execution of the current policy.

2) We use dynamic device parameters to encode node features. Yet, the node features of the previous GNN-based method [18] are composed solely of static technological data, such as threshold voltage and electron mobility. The exclusion of crucial dynamic device parameters from the node features complicates the process of learning the relationships between device parameters and design goals and leads to divergence of learning in our study.

3) We choose the graph attention network (GAT) [33] as the backbone of the GNN part, whereas previous work [18] often applies the graph convolutional network (GCN) to solve the problem. The multihead attention mechanism of GAT helps to learn more complex and higher-dimensional interactions between a circuit node and its neighbors (refer to Fig. 2). Our empirical studies (elaborated in Section V) show that GAT often performs better than other GNNs, such as the GCN [34] in the P2S task. Note that we customize the GAT to model circuit topologies and apply them to the P2S task rather than inventing novel GNN structures. The fundamental operations underlying the proposed GAT follow those of the original publications [33] and, therefore, are omitted here. With this unique policy network architecture, each circuit has its specific state representation: a vector consists of its design specifications (used as the input for the FCNN part) and a matrix with the circuit node feature encoding (used as the input for the GNN part). We perform thorough experimental studies in Section V-A to show that the customized network architecture is superior to other methods.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DESIGN SPACE, SAMPLING SPACE, AND PVT CORNERS FOR FOUR BENCHMARK CIRCUITS

Circuit types	Single-stage Op-Amp	Two-stage Op-Amp	Folded-cascode Op-Amp	Nested Miller compensation Op-Amp
Technology	GlobalFoundries 130/65/28 nm			
16 PVT conditions	Process:{SS, SF, FS, FF}	Voltage:{1.1V, 1.3V}		Temperature:{$-40°C$, $125°C$}
Design space	10^{24} values	10^{31} values	10^{17} values	10^{39} values
Width (nm)	mp1-4:[200, 2000, 10] mn1-3:[2000, 10000, 10]	mp1-2:[1000, 100000, 1000] mn1-4: [1000, 100000, 1000]	mp1-2:[1000, 10000, 200] mn1-3:[160, 1000, 20]	mp1-3&mn4:[10000, 50000, 1000] mp4: [50000, 250000, 10000] mn1-3:[2000, 20000, 1000]
Capacitance (pF)	C_L: 0.12	c:[0.1, 10, 0.1] / C_L: 1	c:[0.1, 10.0, 0.2] / C_L: 1	c1:[25, 50, .5]/c2:[1, 25, .5]/C_L: 100
Sampling space				
Gain (dB) / I (A)	[40,45] / [10^{-5}, 10^{-4}]	[10,20] / [10^{-3}, 10^{-2}]	[20,30] / [10^{-4},10^{-3}]	[40,45] / [10^{-3},10^{-2}]
PM (○) / BW (MHz)	[>50] / [0.5, 1]	[>60] / [1, 20]	[>85] / [4, 6]	[>55] / [1, 2]

E. Optimization Methods for Policy Training

Combining the GAT and FCNN forms the policy network $\pi_\theta(a|s)$ parameterized by $\theta = \{W_{\text{GAT}}, W_{\text{FC}}\}$. Here, W_{GAT}, W_{FC} are the learnable parameters for the GAT and FCNN. Our goal is to make the RL agent gain rich circuit design experiences and generate higher-quality decisions by interacting with the environment. We can formally define the objective function of automated design of analog circuits as follows:

$$J(\theta, G) = 1/H \cdot \sum\nolimits_{g\sim G} \mathbb{E}_{g,s\sim\pi_\theta}\left[R_{s,g}\right]. \tag{4}$$

Here, H is the space size of all desired specifications G and $R_{s,g}$ is the episode reward. Given the cumulative reward for each episode, we use PPO [35] to update the parameters of the policy network with a clipped objective

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_k\left[\min(b_i(\theta), \text{clip}(b_k(\theta), 1-\epsilon, 1+\epsilon))\hat{A}_k\right] \tag{5}$$

where $\hat{\mathbb{E}}_k$ represents the expected value at time step k; b_k is the probability ratio of the new policy and the old policy, and $\hat{A}_k$ is the estimated advantage at time step k.

Previous RL-based methods [17], [18] for P2S tasks mainly explore DDPGs to train RL agents and have shown promising performance. However, the lack of a detailed comparison between different RL algorithms makes it difficult to determine which is better for P2S tasks. DDPG is an off-policy RL method that uses two separate policies for exploration and updates, a stochastic behavior policy for exploration, and a deterministic policy for the target update. The "deterministic" in DDPG refers to the fact that the agent computes the action directly instead of a probability distribution over actions. DDPG is specifically designed for environments with continuous action spaces and continuous state spaces, making it an equally valid choice for continuous control tasks applicable to fields, such as robotics or autonomous driving.

On the other hand, PPO is an on-policy RL method, that is, it involves collecting a small batch of experiences by interacting with the environment according to the latest version of its stochastic policy and using that batch to update its decision-making policy. The "stochastic" in PPO refers to the fact that the agent computes the action as a probability distribution instead of directly over actions. PPO can often work with both discrete and continuous action spaces, making it suitable for a wide range of RL tasks in various domains, e.g., training ChatGPT. In particular, we use RL with discrete action space to build our framework due to: 1) experienced human designers also use fine-grained tuning (i.e., adjusting device parameters with several discrete tuning steps) to tackle the P2S task and 2) the thorough study in Section V-C shows that PPO with discrete action space achieves better performance.

IV. EXPERIMENTAL METHODOLOGY

In this section, we present the experimental methodology for evaluating the proposed framework. First, we introduce the circuit benchmarks used in our evaluations. Then, baselines for comparisons are briefly discussed. Finally, we show the training platform and configurations of our framework.

A. Benchmarks and Performance Metrics for Evaluation

Op-Amps are commonly used as circuit benchmarks in prior art [15], [16], [18], [28], [31], [36] and are also widely used as essential building blocks in many analog subsystems. Therefore, we take multiple Op-Amps to evaluate the proposed framework. In particular, we adopt a single-stage cascode Op-Amp, a two-stage Op-Amp, a folded-cascode Op-Amp [37], and a three-stage nested Miller compensation Op-Amp with feedforward transconductance stage [38] (NMCF) in our benchmark. These circuits have diverse topologies and design complexities. The detailed schematics of these circuits were shown in previous work [15], [16], [18], [28], [31], [36] and are thereby omitted here. The design space for the device parameters, the sampling space for the circuit specifications, and the PVT corners are listed in Table I. There are $4 \times 2 \times 2 = 16$ extreme PVT conditions, including four process variations, two voltage variations, and two temperature variations.

With the circuit benchmark, we examine mainly the sampling efficiency, design success rate, and design efficiency of our framework. We show the sampling efficiency of RoSE-Opt by using a control experiment, that is, to train the RL agent with/without the BO vanguard. The *sampling efficiency* is defined as the number of SPICE simulations saved to achieve the same training quality (i.e., training reward) compared to the control group. To allow a reasonable comparison between different design automation methods, we also propose a $\text{FoM}_{\text{deploy}}$ defined as the ratio between the design success rate (N_{success}) and the design efficiency ($N_{\text{step}} \cdot T_{\text{sim}}$): $\text{FoM}_{\text{deploy}} = N_{\text{success}}/(N_{\text{step}} \cdot T_{\text{sim}})$. Here, N_{success} is the *design success rate* of policy deployment by giving 200 groups of design goals randomly sampled from the specification space. N_{step} is the

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

average number of required deployment steps (i.e., the number of circuit-level simulations) to achieve a group of design goals sampled from the specification space. T_{sim} is the simulation time for each simulation run at the circuit level. Note that the training time for learning-based methods is not included here, as it can be amortized during the deployment phase once the models are well trained (similar to the inference stage in supervised learning).

B. Experiment Platform, Configurations, and Baselines

Our framework is built on Python. We create the circuit graph using the Deep Graph Library [39] and use Ray [40], a well-developed hyperparameter tuning package, to train RL agents. We implement all methods with PyTorch and BoTorch [41]. All experiments were carried out on a 16-core Intel CPU. We train separate RL agents for each circuit. Note that we only need to run BO once at the beginning since we can reuse the starting point optimized by the BO vanguard in each RL's training and deployment phases. To achieve a more reliable and reproducible experiment result, we decided to run our BO vanguard 50 times and choose the starting point with a mean reward to minimize the variation caused by the initial random sampling. To provide detailed performance comparisons of different RL algorithms, we choose PPO [35] and DDPG [42] as two representatives of the study and also use their default configurations to train policy networks.

Although various previous methods have been proposed to address P2S tasks, such as BO [1], genetic algorithm [13], and RL methods [18], [28], they do not consider PVT variations and post-layout parasitics in the optimization process of device parameters. As a result, we modify all of these previous works to include all PVT corners for a direct comparison. We also compare to the most recent work, RobustAnalog [31], which solves the P2S task considering the effect of partial variations in PVT. Despite several major differences between RobustAnalog and RoSE-Opt, we care most about the efficacy of RobustAnalog in robust design, as it uses task pruning with reduced PVT corners for RL training, while our RL backbone considers all PVT corners. We follow this strategy to implement RobustAnalog by modifying our RL backbone.

V. Experimental Evaluations

In this section, we show the evaluation results and compare the performance of our proposed framework with previous methods. First, we present the efficacy of incorporating domain knowledge into the learning loop. Second, we show our framework's sampling efficiency and robustness against PVT variations. Third, we show our framework's capability to achieve reliable device sizing by taking into account post-layout parasitics. Fourth, we show how the trained RL agent of our framework assists human designers in finding optimized device parameters, even if it fails in deployment in some cases. Fifth, we present the performance of different RL algorithms in training RL agents for the P2S task. Finally, we also show the performance of our proposed method in Pareto optimization. The section ends with a summarization of comprehensive comparisons between our work and the prior arts.

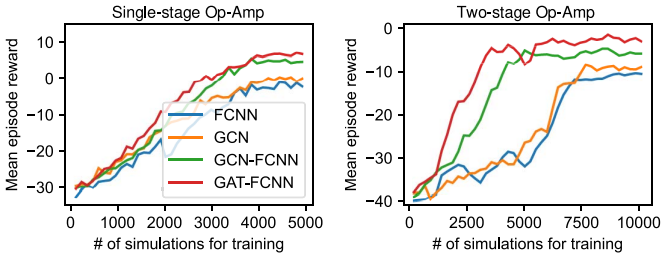


Fig. 3. Comparison between our customized policy network and prior methods' policy networks (GCN [18] and FCNN [28]) for the variation-aware P2S task.

A. Efficacy, Robustness, and Sampling Efficiency

1) Efficacy of Incorporating Domain Knowledge and Customizing Multimodal Policy Network: We conducted comprehensive experimental studies to show the effectiveness of incorporating domain knowledge and customizing a multimodal policy network. First, we present the importance of integrating domain knowledge into the learning process (e.g., taking PVT variations as an example). Existing methods [18], [28] often overlook PVT variations and parasitic effects in the layout when designing reward functions, leading to failures during deployment in variable environments. In contrast, our proposed framework with a tailored variation-aware reward function achieves a design success rate that exceeds 90% in nonideal environments with PVT variations as listed in Table III. Second, we show the superiority of customizing a multimodal policy network. We compared different policy networks used by previous methods [18], [28] with a consistent variation-aware reward function. Specifically, the work [28] uses a simple FCNN as a policy network and does not exploit circuit topologies. The work [18] introduces a GCN policy network based on a partial circuit topology and uses static semiconductor technology information as observations. Our study reveals that the GCN-based network could experience training divergence, necessitating adjustments based on our methodology (e.g., using node features with dynamic device parameters) for a reliable comparison. The comparisons between different methods are illustrated in Fig. 3. Our customized GNN-FCNN policy network demonstrates superior performance, yielding higher rewards with the two circuits. In particular, the GAT-FCNN-based policy surpasses the GCN-FCNN-based policy, highlighting the effectiveness of our proposed policy network architecture. Unless otherwise specified, GAT-FCNN will be the default learning model for the remainder of this section, generally referred to as GNN-FCNN. These experimental studies demonstrate the effectiveness of integrating domain knowledge and tailoring a multimodal policy network.

2) Robust Design With Full PVT Incorporation: We then show the robust design enabled by our RL backbone against PVT variations by deploying our RL agent in an environment taking full account of PVT variations. Policy deployment applies a trained policy to automatically find the device parameters for given design goals. The left column of Fig. 4 shows the deployment trajectories under several representative PVT corners by taking the PM of the Folded-cascode Op-Amp as an example, where each color represents a PVT corner.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

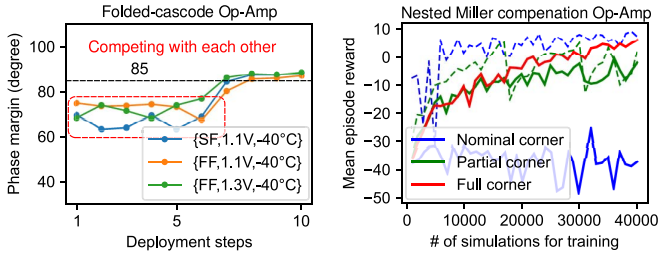


Fig. 4. Left: Illustration of the competing phenomenon between PVT corners. Right: Comparison between different levels of PVT incorporation for RL backbone training. Dashed lines are the actual nominal/partial corner training curve. Solid lines are the training curve evaluated under full PVT corners.

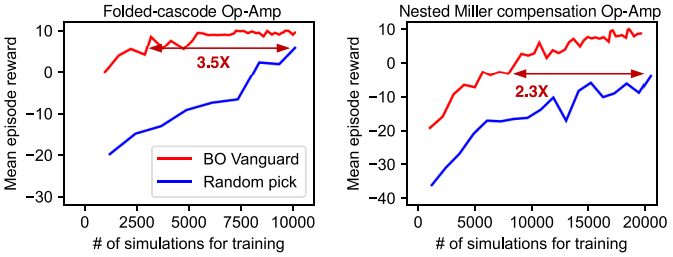


Fig. 5. Example to show the comparison of the training sample efficiency by using the RL Backbone with or without preoptimization of its starting state.

It can be seen that although each trajectory under a specific PVT corner is smooth, the worst corner can be quickly replaced by another corner due to the competition between different corners. Here, the worst case indicates the corner where the circuit specification deviates the most from the design goal. This phenomenon shows that device sizing with PVT variations is much more complex compared to the nominal case. Notably, by incorporating PVT variations into our method, our RL agent can achieve a robust design by finding optimal device parameters that can satisfy the design goal in all PVT corners.

To investigate the impact of this corner competition on actual RL training, we train three RL backbones with different levels of PVT incorporation. As shown in the right column of Fig. 4, although the RL backbone trained with the nominal corner (or partial PVT corner) achieves convergence with high reward, it completely (or partially) diverges when deployed in an environment with full PVT variations (i.e., the dashed curves). The result verifies that device sizing with full PVT variations is much more complex compared to the nominal case. It also suggests that previous work, RobustAnalog [31], which uses the K-means clustering to prune the PVT corners during the training process, is not sufficient to ensure robustness in an environment with full variations of PVT. It can delay the training process because the RL algorithm needs more training steps to repeat the clustering whenever the device parameters reach the design goal under partial PVT corners, but fail in the setup of full PVT corners.

3) Efficient Sampling With BO Vanguard: Third, we show that the BO vanguard can improve sampling efficiency to train our RL backbone. Fig. 5 illustrates the example training curves of our RL backbone to design two types of Op-Amps with two different starting points, one is from BO searching (labeled as "BO Vanguard") and the other is a randomly selected

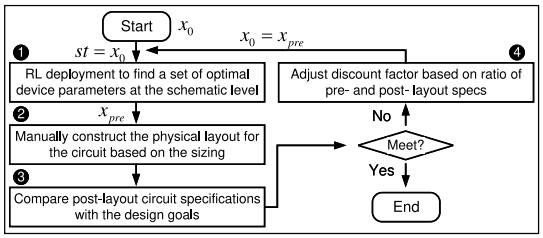


Fig. 6. Flow RoSE-Opt used for the parasitic-aware sizing. Please refer to Fig. 1 for the parasitic aware experiment's schematic and layout.

value from the device parameter design space S_P, e.g., median value (labeled as "Random pick"). It shows that the RL agent without an optimized starting point often needs more circuit-level simulations to achieve the same reward as the one with an optimized starting point from BO (e.g., in this case, 3.5× for Folded-cascode Op-Amp and 2.3× for three-stage nested Miller compensation Op-Amp). Thus, by optimizing the starting point, the RL agent converges faster with fewer sampling data (that is, fewer circuit-level simulations).

B. Parasitic-Aware Device Parameter Optimization

We continue to study how to apply RoSE-Opt to optimize parasitic-aware device parameters. Without considering the parasitic effect of physical layouts at the prelayout design stage, the obtained device parameters cannot guarantee the circuit specifications after the schematic is directly transferred into a physical design. In practice, there are often tens of iterations between schematic design and physical design performed by human designers to fine-tune the device parameters to ensure that the circuit under design meets the design goals.

Several previous works have explored learning-based methods to address this parasitic-aware optimization problem [28], [43], [44]. An early RL-based method [28] aims to tackle it by deploying the trained RL agent in a parasitic-aware environment. In particular, this method uses the BAG tool [10] to automatically generate a physical layout for the circuit based on the device parameters at each deployment step and exploits the reward from post-layout simulation to guide the search process for the trained RL agent. This process continues until the agent meets the target when parasitics are considered or it has reached the maximum allocated deployment steps. Another work [43] attempts to tackle the problem by combining supervised learning and BO. The idea is to train a graph neural network to predict the parasitics of an analog circuit with its device parameters and then back-annotate the parasitics to the circuit schematic. With this processing step, BO is applied to search for optimal device parameters through the parasitic-aware schematic.

We explore another method to solve the same problem with much higher flexibility. Our method relies on two key observations from the human design loop. First, human experts often construct an initial physical layout of the circuit with an initial set of device parameters and fine-tune the device parameters by following the same placement of the device as the one used in the initial physical layout. Second, circuit specifications from the post-layout simulation of this initial physical layout are often degraded compared to the desired goals but are not far from them. Therefore, the optimal final

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF DEVICE PARAMETERS BEFORE AND AFTER PHYSICAL DESIGN WITH DESIGN GOALS OF GAIN = 45 DB, BW = 800 KHZ, PM = 50°, AND POWER = 15 μW

Device	mp1	mp2	mp3	mp4	mn1	mn2	mn3	Gain	BW	PM	Power
Schematic design	1.02 μm	1.02 μm	1.02 μm	1.02 μm	6.2 μm	4.94 μm	6.2 μm	47.3 dB	862 kHz	52°	12 μW
Layout design	1.02 μm	1.02 μm	1.02 μm	1.02 μm	5.26 μm	5.26 μm	5.26 μm	46.91 dB	897 kHz	53.52°	12 μW

device parameters to meet the design goals also fall in the neighborhood of the initial set of device parameters.

With these key observations in mind, our method can apply to parasitic-aware device parameter optimization by following the essential steps shown in Fig. 6. We begin by initializing all discount factors to 1. These discount factors are explained in ❹. The other steps are as follows:

❶ Deploy the trained RL agent to find the set of optimal device parameters that satisfy the design goals in the prelayout stage and perform simulations to obtain the circuit specifications with this set of device parameters; the ith specification of the jth corner is marked as $s^j_{i,\text{pre}}$.

❷ Construct a physical layout with the device parameters found in Step ❶.

❸ Extract the circuit specifications (i.e., $s^j_{i,\text{post}}$) of this physical layout by performing a post-layout simulation and compare them with the design goals; if satisfied, the design is successful; otherwise, jump to Step ❹.

❹ Adjust the discount factor based on the ratio between pre- and post- layout specifications, e.g., if $s^j_{i,\text{pre}} \geq s^j_{i,\text{post}}$, $\alpha^j_i = s^j_{i,\text{post}}/s^j_{i,\text{pre}}$.

After the first iteration, we repeat the flow using the set of device parameters x_{pre} found in Step ❶ as the new starting point st of the trained RL agent for another deployment and the intermediate circuit specification in ❶ will be discounted by the discount factor as $\alpha^j_i \cdot s^j_{i,\text{pre}}$, until the optimal final device parameters that satisfy the design goals in the prelayout stage are found. This method essentially follows a dynamic over-design strategy that can alleviate the challenge of estimating a static over-design value derived from domain knowledge for complex circuits. Our experiments with the single-stage Op-Amp illustrated in Fig. 1 show that our method generally takes no more than two rounds to reach a set of device parameters with which the circuit specifications of a physical layout can also meet the design goal. The final physical layout is similar to that shown in Fig. 1 and is omitted here.

C. Comparisons Between Different RL Algorithms

Different RL algorithms have shown different performance in solving practical problems. PPO and DDPG are two primary RL algorithms used in current RL-based methods [17], [18], [28] for P2S tasks. Here, we perform detailed experiments to compare their performance in tackling the P2S task by using the design of a two-stage Op-Amp as an example. Fig. 7 illustrates our evaluation results, where we train RL agents with PPO using both "discrete" and "continuous" actions, as well as DDPG with continuous action. Each curve in Fig. 7 is based on 6 random seeds with 3×10^6 simulations budget for training. All the RL algorithms perform batch training by collecting 6 simulation results in parallel.

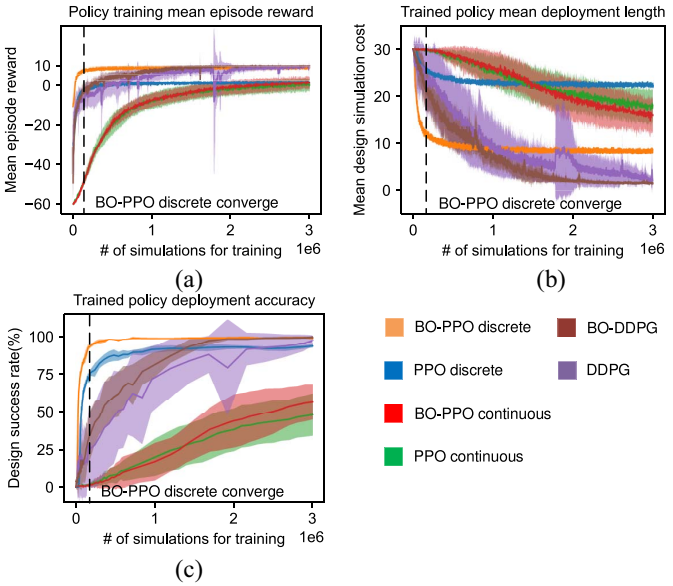


Fig. 7. Comparisons between DDPG and PPO when applied to train RL agents and deploy trained checkpoints for 200 randomly sampled design goals. (a) Mean episode reward for training, (b) mean deployment simulation cost, and (c) mean design success rate. Each curve is based on six random seeds. The vertical dashed line is the convergent point for BO-PPO discrete (our method).

Note that we also observe similar results for other types of Op-Amps.

1) PPO-Continuous Versus DDPG-Continuous: Compared to DDPG-continuous, we find that PPO-continuous has a lower-sample efficiency during the training process. This is because PPO-continuous adopts an on-policy learning mechanism that samples actions according to its latest stochastic policy. The on-policy characteristic introduces variance, since each estimate of an expectation over a finite set of samples may vary, which necessitates a large number of samples for accurate mean calculations, thereby leading to low-sampling efficiency. In contrast, DDPG-continuous utilizes an off-policy learning mechanism, which involves a replay buffer to store transitions from the previous policy and relies on the current policy only to replenish the buffer, improving the sampling efficiency. The lower-sampling efficiency of PPO-continuous also impacts the training quality of the policy. As shown in Fig. 7, with the same number of training samples, the mean design simulation cost (design accuracy) of the trained policy with PPO-continuous (green line) is higher (lower) than that of the one with DDPG-continuous (red line).

2) PPO-Discrete Versus DDPG-Continuous: With the setting of discrete action space, PPO-discrete demonstrates superior training sample efficiency and more consistent results compared to its continuous counterpart and DDPG-continuous. In a discrete environment, the action choices of an RL

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

agent at each step are simplified to adjust the parameter upward/downward with a small increment/decrement, or to maintain its current value. This simplicity makes the training process smoother and improves the quality of the policy to find the optimal device parameters during the deployment process, as shown in Fig. 7 (blue line). In a complex continuous environment where the agent has a plethora of parameter choices, the off-policy mechanism of DDPG-continuous could suffer from biases, where some of the updates are based on prior (potentially incorrect) expectation estimates. This leads to irreversible incorrect estimates in the end and causes training instability due to its inherent low-variance but high-bias nature. As shown in Fig. 7 (red line), there is a sudden change with respect to the mean episode reward and design accuracy/cost.

3) BO-PPO-Discrete Versus PPO-Discrete: To address the issue of the low-design efficiency of PPO-discrete, we introduce BO to optimize the starting point of PPO-discrete, thus positioning the initial state closer to the solution space. As illustrated in Fig. 7 (yellow line), this strategy could optimize the starting point, allowing PPO-discrete to reach the desired specifications without too much meaningless exploration, accelerating the trajectory formulation toward the solution. Ultimately, this integration of BO with PPO-discrete, termed BO-PPO discrete, demonstrates superior performance in both design accuracy and sampling efficiency, and achieves commendable results in design efficiency. For other continuous BO-RL algorithms (i.e., BO-DDPG-continuous versus DDPG-continuous), we also observe performance improvement compared to its RL baseline by producing more consistent results. This novel approach showcases the potential of combining classical optimization techniques to improve the effectiveness of RL in complex analog circuit design. As optimization techniques continue to evolve, the combined strengths of different optimization methods, such as BO and RL, underscore the importance of hybrid strategies in complex optimization scenarios of analog circuit parameters.

D. Comparisons Between Different Methods and Summary

Finally, we compare our proposed RoSE-Opt method with previous RL methods [18], [28], [31] and conventional optimization methods, such as BO [1] and genetic algorithm [13]. In addition to a general variation-aware device sizing task that we introduced in Section III, we also study a specific task of analog design, i.e., Pareto optimization. The goal is to optimize a particular circuit specification or a FoM that consists of multiple circuit specifications [45]. Here, we use the standard FoM of Op-Amp defined as $\text{FoM}_{\text{Op-Amp}}(\text{MHz}\cdot\text{pF}/\,\mu\text{W}) = (\text{GBW}\cdot C_L)/P$ to customize a reward function and train RL agents (or use BO/Genetic Algorithm) to maximize it. Note that we also observe similar results for the nominal corner experiment.

1) Variation-Aware Device Sizing: Fig. 8 illustrates the results. With the benchmark circuit, RoSE-Opt can achieve a design success rate of 96.5% and an average deployment length of 12 simulation costs within 20000 simulations for training. In contrast, with the same number of simulations, the

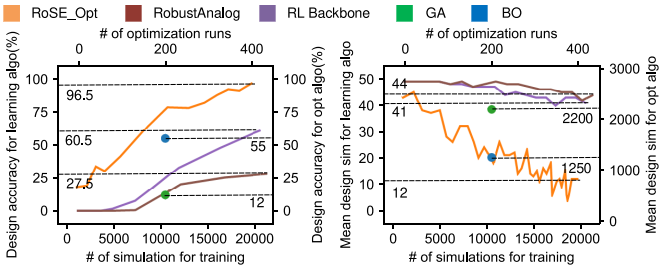


Fig. 8. Comparisons between our proposed framework RoSE-Opt, our RL Backbone, RobustAnalog [31], BO [1], and GA [13] for variation-aware device sizing problem by taking the design of nested Miller compensation Op-Amp as an example. Left: Mean design success rate. Right: Mean design simulation cost.

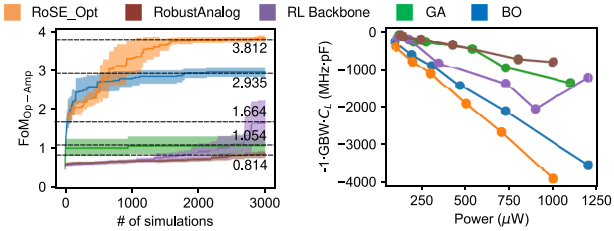


Fig. 9. Comparisons between our proposed framework RoSE-Opt, our RL Backbone, RobustAnalog [31], BO [1], and GA [13] for Pareto optimization problem by taking the design of nested Miller compensation Op-Amp as an example. Left: $\text{FoM}_{\text{Op-Amp}}$ result. Right: Pareto frontier. Each curve is based on six random seeds.

RL Backbone attains a lower-design success rate and a longer deployment length. The comparison again shows that the BO vanguard indeed improves sample efficiency. RobustAnalog is even less competitive to the RL Backbone due to the incorporation of only partial PVT variations into the learning framework. Traditional ad-hoc optimization methods, such as BO and GA, cannot solve the variation-aware device sizing problem with high accuracy and efficiency. Thus, our RoSE-Opt framework achieves the best-sampling efficiency, design efficiency, and design success rate by considering all PVT variations in the learning loop and adopting the two-level optimization method at the same time.

2) Pareto Optimization: Fig. 9 shows the Pareto frontier of different methods. Compared to baselines, our RoSE-Opt achieves the highest $\text{FoM}_{\text{Op-Amp}}$ and defines the optimal Pareto frontier by achieving the lowest-power consumption while maintaining the highest GBW within the fewest simulations.

In the end, we summarize RoSE-Opt together with previous comparisons in Table III. In particular, we use BO [1] and GA [13] as representatives of prior optimization-based methods and use RobustAnalog [31], RL1 [28], and RL2 [18] as representatives of previous learning-based methods. Additionally, we use the $\text{FoM}_{\text{deploy}}$ defined previously to evaluate the overall performance of a design automation method in the P2S problem and use the $\text{FoM}_{\text{Op-Amp}}$ for Pareto optimization evaluation. The comparisons show that our proposed RoSE-Opt framework, which uses BO as preoptimization and a domain-knowledge-infused RL agent as the main optimization backbone, can significantly improve training sample efficiency, design efficiency, and design success rate for the PVT-aware design and improve Pareto optimization. Note that BO is performed only

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

TABLE III
SUMMARY OF COMPARISON WITH EXISTING LEARNING AND OPTIMIZATION METHODS

Methods	Domain knowledge		Starting point optimization	P2S problem				$FoM_{Op\text{-}Amp}$ opt problem
				Training sample efficiency		FoM_{deploy}		$FoM_{Op\text{-}Amp}$ value
	Multimodal	PVT		folded-cascode	NMCF	folded-cascode	NMCF	NMCF
RobustAnalog[a] [31]	Yes	Partial	No	1×	1×	1.14	0.625	0.814
BO[b] [1]	No	Full	N/A[c]	N/A[c]	N/A[c]	0.46	0.044	2.935
GA[b] [13]	No	Full	N/A[c]	N/A[c]	N/A[c]	0.067	0.005	1.054
RL1[b] [28]	No	Full	No	1.3×	1.32×	1.24	0.9	1.378
RL2[b] [18]	No	Full	No	1.35×	1.4×	1.25	1.1	1.436
RL Backbone	Yes	Full	No	2×	2.5×	1.42	1.26	1.664
RoSE-Opt	Yes	Full	**Yes**	**12**×	**7.9**×	**20.51**	**6.87**	**3.812**

[a] Original work [31] only uses normal 4-layer multilayer perceptions. We modify its network to the same multi-modality network we used in our RL backbone.
[b] Previous methods [1], [18], [28], [13] without considering PVT variations fail in the robust design with zero design accuracy and close to 0 $FoM_{Op\text{-}Amp}$. As a result, we modify their work to include full PVT incorporation.
[c] Traditional optimization methods [1], [13] do not need pre-optimized starting points and training (BO [1] is still considered as an ad-hoc optimization method since it only uses active learning for improving its optimization sample efficiency without generalization to unseen design goals). Thus, some metrics are not applicable here.

once at the very beginning, thereby incurring minimal overhead. In summary, our knowledge-infused RoSE-Opt framework that takes advantage of the complementary benefits of learning and optimization algorithms (i.e., combining BO and RL) can achieve the best FoM for the challenging reliable device sizing problem.

VI. CONCLUSION

We propose a BO-RL-based framework to automate the P2S task for analog circuit design, by incorporating domain knowledge (e.g., circuit topology, specification tradeoffs, PVT variations, layout parasitics) for design robustness and leveraging BO to enhance RL's starting point for training efficiency. We show that such a framework is superior in designing various analog circuits with higher accuracy, efficiency, and reliability. We expect that our method would assist human designers in accelerating analog chip design with artificial agents that master massive circuitry optimization experiences via learning.

REFERENCES

[1] W. Lyu, F. Yang, C. Yan, D. Zhou, and X. Zeng, "Batch Bayesian optimization via multi-objective acquisition ensemble for automated analog circuit design," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 3306–3314.

[2] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, Jun. 2021.

[3] B. Khailany et al., "Accelerating chip design with machine learning," *IEEE Micro*, vol. 40, no. 6, pp. 23–32, Dec. 2020.

[4] B. Razavi, *RF Microelectronics* (Prentice Hall Communications Engineering and Emerging Technologies Series), 2nd ed. Saddle River, NJ, USA: Prentice Hall, 2011.

[5] S. Datta, "Ten nanometre CMOS logic technology," *Nat. Electron.*, vol. 1, no. 9, pp. 500–501, Sep. 2018.

[6] W. Cao, X. He, A. Chakrabarti, and X. Zhang, "NeuADC: Neural network-inspired RRAM-based synthesizable analog-to-digital conversion with reconfigurable quantization support," in *Proc. Design, Autom. Test Eur. Conf. Exhibit. (DATE)*, 2019, pp. 1477–1482.

[7] W. Cao, L. Ke, A. Chakrabarti, and X. Zhang, "Neural network-inspired analog-to-digital conversion to achieve super-resolution with low-precision RRAM devices," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2019, pp. 1–7.

[8] W. Cao, Y. Zhao, A. Boloor, Y. Han, X. Zhang, and L. Jiang, "Neural-PIM: Efficient processing-in-memory with neural approximation of peripherals," *IEEE Trans. Comput.*, vol. 71, no. 9, pp. 2142–2155, Sep. 2022.

[9] G. Zhang, H. He, and D. Katabi, "Circuit-GNN: Graph neural networks for distributed circuit design," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 7364–7373.

[10] J. Crossley et al., "BAG: A designer-oriented integrated framework for the development of AMS circuit generators," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2013, pp. 74–81.

[11] M. Degrauwe et al., "IDAC: An interactive design tool for analog CMOS circuits," *IEEE J. Solid-State Circuits*, vol. 22, no. 6, pp. 1106–1116, Dec. 1987.

[12] R. Harjani, R. Rutenbar, and L. Carley, "OASYS: A framework for analog circuit synthesis," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 8, no. 12, pp. 1247–1266, Dec. 1989.

[13] B. Liu et al., "Analog circuit optimization system based on hybrid evolutionary algorithms," *Integration*, vol. 42, no. 2, pp. 137–148, 2009.

[14] G. Wolfe and R. Vemuri, "Extraction and use of neural network models in automated synthesis of operational amplifiers," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 22, no. 2, pp. 198–212, Feb. 2003.

[15] Y. Li, Y. Wang, Y. Li, R. Zhou, and Z. Lin, "An artificial neural network assisted optimization system for analog design space exploration," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2640–2653, Oct. 2020.

[16] M. V. Harsha and B. P. Harish, "Artificial neural network model for design optimization of 2-stage op-amp," in *Proc. 24th Int. Symp. VLSI Design Test (VDAT)*, 2020, pp. 1–5.

[17] Z. Zhao and L. Zhang, "Deep reinforcement learning for analog circuit sizing," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2020, pp. 1–5.

[18] H. Wang et al., "GCN-RL circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning," in *Proc. 57th ACM/EDAC/IEEE Design Autom. Conf.*, 2020, pp. 1–6.

[19] J. Gao, W. Cao, and X. Zhang, "RoSE: Robust analog circuit parameter optimization with sampling-efficient reinforcement learning," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, 2023, pp. 1–6.

[20] A. Ghaffari, M. Asgharian, and Y. Savaria, "Statistical hardware design with multi-model active learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 2, pp. 562–572, Feb. 2024.

[21] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. Wong, "BOOM-explorer: RISC-V BOOM microarchitecture design space exploration framework," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2021, pp. 1–9.

[22] S. Liu, F. C. Lau, and B. C. Schafer, "Accelerating FPGA prototyping through predictive model-based HLS design space exploration," in *Proc. 56th Annu. Design Autom. Conf.*, 2019, pp. 1–6.

[23] V. Mnih et al., "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.

[24] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 3389–3396.

[25] S. Yun, J. Choi, Y. Yoo, K. Yun, and J. Young Choi, "Action-decision networks for visual tracking with deep reinforcement learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1349–1358.

[26] T. Young, D. Hazarika, S. Poria, and E. Cambria, "Recent trends in deep learning based natural language processing [review article]," *IEEE Comput. Intell. Mag.*, vol. 13, no. 3, pp. 55–75, Aug. 2018.

[27] A. Mirhoseini et al., "Chip placement with deep reinforcement learning," 2020, *arXiv:2004.10746*.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.

[28] K. Settaluri, Z. Liu, R. Khurana, A. Mirhaj, R. Jain, and B. Nikolic, "Automated design of analog circuits using reinforcement learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 9, pp. 2794–2807, Sep. 2022.
[29] D. M. Colleran et al., "Optimization of phase-locked loop circuits via geometric programming," in *Proc. IEEE Custom Integr. Circuits Conf.*, 2003, pp. 377–380.
[30] M. Hershenson, S. Boyd, and T. H. Lee, "Optimal design of a CMOS op-amp via geometric programming," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 20, no. 1, pp. 1–21, Jan. 2001.
[31] W. Shi et al., "RobustAnalog: Fast variation-aware analog circuit design via multi-task RL," in *Proc. ACM/IEEE Workshop Mach. Learn. CAD*, 2022, pp. 35–41.
[32] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
[33] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–12.
[34] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–14.
[35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[36] Y. Li et al., "A circuit attention network-based actor-critic learning approach to robust analog transistor sizing," in *Proc. ACM/IEEE 3rd Workshop Mach. Learn. CAD (MLCAD)*, 2021, pp. 1–6.
[37] B. Razavi, *Design of Analog CMOS Integrated Circuits*, 1st ed. New York, NY, USA: McGraw-Hill, Inc., 2000.
[38] K. N. Leung and P. Mok, "Analysis of multistage amplifier-frequency compensation," *IEEE Trans. Circuits Syst. I, Fundam. Theory Appl.*, vol. 48, no. 9, pp. 1041–1056, Sep. 2001.
[39] M. Wang et al., "Deep graph library: A graph-centric, highly-performant package for graph neural networks," 2020, *arXiv:1909.01315*.
[40] P. Moritz et al., "Ray: A distributed framework for emerging AI applications," in *Proc. 13th USENIX Conf. Oper. Syst. Design Implement.*, 2018, pp. 561–577.
[41] M. Balandat et al., "BoTorch: A framework for efficient monte-carlo Bayesian optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1–15.
[42] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
[43] M. Liu, W. J. Turner, G. F. Kokai, B. Khailany, D. Z. Pan, and H. Ren, "Parasitic-aware analog circuit sizing with graph neural networks and Bayesian optimization," in *Proc. Design, Autom. Test Eur. Conf. Exhibit. (DATE)*, 2021, pp. 1372–1377.
[44] A. F. Budak, K. Zhu, and D. Z. Pan, "Practical layout-aware analog/mixed-signal design automation with Bayesian neural networks," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2023, pp. 1–8.
[45] W. Cao, M. Benosman, X. Zhang, and R. Ma, "Domain knowledge-infused deep learning for automated analog/radio-frequency circuit parameter optimization," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, 2022, pp. 1015–1020.



Weidong Cao (Member, IEEE) received the Ph.D. degree from Washington University in St. Louis, St. Louis, MO, USA, in 2021.

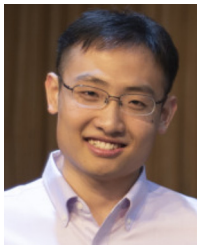
He is an Assistant Professor with the Department of Electrical and Computer Engineering, The George Washington University, Washington, DC, USA. Before that, he was a Principal Engineer with TSMC Corporate Research, San Jose, CA, USA. His research interests focus on VLSI design, computer architecture, EDA, and quantum computing.

Dr. Cao is the recipient of IEEE/ACM ISLPED Best Paper Award in 2022 and his work has also been nominated for Best Paper Award at DATE 2019 and DAC 2022.



Jian Gao (Student Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Washington University in St. Louis, St. Louis, MO, USA, in 2023. He is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, Northeastern University, Shenyang, China.

His research interest lies in design automation for analog and mixed-signal circuits.



Tianrui Ma (Student Member, IEEE) received the B.S. (Hons.) degree in electrical engineering from Beihang University, Beijing, China, in 2019, and the M.S. degree in electrical engineering from Washington University in St. Louis, St. Louis, MO, USA, in 2023, where he is currently pursuing the Ph.D. degree.

He is interested in systematic design and architectural optimization of CMOS image sensor-based vision systems.

Mr. Ma is a recipient of the ISLPED 2022 Best Paper Award.



Rui Ma (Senior Member, IEEE) received the Dr.-Ing. degree in electrical engineering from Kassel University, Kassel, Germany, in 2009.

He is a Director of mmWave PA systems with Psemi A muRata Company, San Diego, CA, USA. He was a Senior Principal Research Scientist with Mitsubishi Electric Research Scientist, Cambridge, MA, USA, before joining Psemi in 2022.

Dr. Ma is an IEEE Distinguished Microwave Lecturer and was an Associate Editor of IEEE TRANSACTION ON MICROWAVE THEORY AND TECHNOLOGY.



Mouhacine Benosman (Senior Member, IEEE) received the Ph.D. degree in applied mathematics from Ecole Centrale de Nantes, Nantes, France, in 2002.

He is a Senior Principal Research Scientist with the Mitsubishi Electric Research Laboratories, Cambridge, MA, USA. His research interests include nonlinear robust and fault tolerant control, multiagent control with applications to robotics, estimation and control of partial differential equations with applications to thermo-fluid models, learning-based adaptive control for nonlinear systems, control-theory based optimization algorithms, and physics informed machine learning.

Dr. Benosman currently serves as the Senior Editor for the INTERNATIONAL JOURNAL OF ADAPTIVE CONTROL AND SIGNAL PROCESSING.



Xuan Zhang (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from Cornell University, Ithaca, NY, USA, in 2012.

She is an Associate Professor with the Electrical and Computer Engineering Department, Northeastern University, Shenyang, China. She works across the fields of integrated circuits/VLSI design, computer architecture, and electronic design automation.

Dr. Zhang is the recipient of NSF CAREER Award in 2020. Her work has received numerous best paper awards and nominations, including the ISLPED Best Paper Award in 2022, the AsianHOST Best Paper Award in 2020, the DATE Best Paper Award in 2019, and the Nomination for Best Paper Awards at DAC 2022, ASP-DAC 2021, MLCAD 2020, DATE 2019, and DAC 2017. She is an IEEE Women in Engineering Distinguished Lecturer from 2023 to 2024, IEEE Circuits and Systems Society Distinguished Lecturer from 2022 to 2023. She currently serves as the Associate Editor-in-Chief for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I and an Associate Editor for IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGNS.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on June 22,2026 at 17:57:24 UTC from IEEE Xplore. Restrictions apply.